Introduction to Stored Procedures

A stored procedure is a named, precompiled collection of one or more SQL statements that is stored inside the database and can be executed (called) whenever required. Instead of sending a long block of SQL from an application every time, the application simply calls the procedure by name.

Stored procedures behave like functions or subroutines in a programming language — they can accept parameters, perform logic (loops, conditions, variable declarations), and return results.

Why use Stored Procedures?

- **Performance: Precompiled and cached by the database engine, so repeated execution is faster than sending raw SQL each time.**
- **Reusability: Write the logic once, call it from many applications or scripts.**
- **Security: Users can be granted EXECUTE permission on a procedure without granting direct access to the underlying tables.**
- **Reduced network traffic: A single CALL statement replaces many lines of SQL sent over the network.**
- **Maintainability: Business logic is centralized in the database, so changes are made in one place.**
- **Consistency: Ensures the same validation/business rules are applied every time, regardless of which application calls it.**

2. CREATE PROCEDURE Syntax

The general syntax (MySQL dialect) to create a stored procedure is:

```
DELIMITER //

CREATE PROCEDURE procedure_name (
        [IN | OUT | INOUT] parameter_name data_type, ...
)
```

BEGIN

-- SQL statements / logic

-- declarations, conditions, loops, etc.

END //

DELIMITER ;

Key points:

- **DELIMITER // temporarily changes the statement terminator from ';' to '//' so that the semicolons inside the procedure body do not end the CREATE PROCEDURE statement prematurely.**
- **The procedure body is enclosed between BEGIN and END.**
- **Parameters are optional — a procedure can be created with zero, one, or many parameters.**
- **After the procedure is created, DELIMITER is reset back to ';'.**

Advantages

Faster execution after compilation.

Reduced duplicate SQL.

Supports transactions, loops, conditions, variables, cursors, and exception handlers.

Easy maintenance.

Disadvantages

Debugging can be difficult.

Vendor-specific syntax.

Complex procedures may become hard to maintain.

3. IN, OUT, and INOUT Parameters

Parameter Type	Direction	Behaviour

IN	**Caller -> Procedure**	**Default type. Passes a value INTO the procedure. Changes made inside do NOT reflect back to the caller.**
OUT	**Procedure -> Caller**	**Procedure sets a value that is returned OUT to the caller. Initial value passed in is ignored.**
INOUT	**Both directions**	**Caller passes a value IN, procedure can modify it, and the updated value is returned back OUT.**

NoParameters

==========

1. Display All Employees

Question:

Write a stored procedure to display all employee records.

Answer:

```
DELIMITER //


CREATE PROCEDURE GetAllEmployees()
BEGIN
   SELECT * FROM Employees;
END //


DELIMITER ;
```

Call

```
CALL GetAllEmployees();
SHOW PROCEDURE STATUS WHERE Db = DATABASE();
```

---

2. Display IT Department Employees

Question:

Write a procedure to display employees working in the IT department.

Answer

```
DELIMITER //

CREATE PROCEDURE GetITEmployees()
BEGIN
   SELECT * FROM Employees
   WHERE department='IT';
END //

DELIMITER ;
Call
CALL GetITEmployees();
```

---

3. Display Employees with Salary Greater Than 60000

Question:

Write a procedure to display employees whose salary is greater than 60000.

Answer

```
DELIMITER //

CREATE PROCEDURE HighSalaryEmployees()
BEGIN
   SELECT * FROM Employees
   WHERE salary > 60000;
END //

DELIMITER ;
```

Call

```
CALL HighSalaryEmployees();
```

---

4. Display Employees from Hyderabad

Question:

Write a procedure to display employees from Hyderabad.

Answer

```
DELIMITER //

CREATE PROCEDURE HyderabadEmployees()
BEGIN
   SELECT * FROM Employees
   WHERE city='Hyderabad';
END //

DELIMITER ;
```

Call

```
CALL HyderabadEmployees();
```

---

5. Display Employees with Experience Greater Than 5 Years

Question:

Write a procedure to display employees having more than 5 years of experience.

Answer

```
DELIMITER //

CREATE PROCEDURE ExperiencedEmployees()
BEGIN
   SELECT * FROM Employees
```

```
   WHERE experience > 5;
END //

DELIMITER ;
Call
CALL ExperiencedEmployees();
```

---

6. Display Employee Names Only

Question:

Write a procedure to display only employee names.

Answer

```
DELIMITER //

CREATE PROCEDURE EmployeeNames()
BEGIN
   SELECT emp_name
   FROM Employees;
END //

DELIMITER ;
Call
CALL EmployeeNames();
```

---

7. Count Total Employees

Question:

Write a procedure to count the total number of employees.

Answer

```
DELIMITER //
```

```
CREATE PROCEDURE TotalEmployees()
BEGIN
   SELECT COUNT(*) AS TotalEmployees
   FROM Employees;
END //

DELIMITER ;
Call
CALL TotalEmployees();
```

---

8. Find Maximum Salary

Question:

Write a procedure to display the maximum salary.

Answer

```
DELIMITER //

CREATE PROCEDURE MaximumSalary()
BEGIN
   SELECT MAX(salary) AS HighestSalary
   FROM Employees;
END //

DELIMITER ;
Call
CALL MaximumSalary();
```

---

9. Find Minimum Salary

Question:

Write a procedure to display the minimum salary.

Answer

```
DELIMITER //

CREATE PROCEDURE MinimumSalary()
BEGIN
   SELECT MIN(salary) AS LowestSalary
   FROM Employees;
END //

DELIMITER ;
Call
CALL MinimumSalary();
```

---

10. Calculate Average Salary

Question:

Write a procedure to calculate the average salary.

Answer

```
DELIMITER //

CREATE PROCEDURE AverageSalary()
BEGIN
   SELECT AVG(salary) AS AverageSalary
   FROM Employees;
END //

DELIMITER ;
```

Call

```
CALL AverageSalary();
```

---

11. Display Employees Ordered by Salary

Question:

Write a procedure to display employees in ascending order of salary.

Answer

```
DELIMITER //

CREATE PROCEDURE EmployeesBySalary()
BEGIN
   SELECT *
   FROM Employees
   ORDER BY salary;
END //

DELIMITER ;
```

Call

```
CALL EmployeesBySalary();
```

---

12. Display Employees Ordered by Experience

Question:

Write a procedure to display employees sorted by experience in descending order.

Answer

```
DELIMITER //

CREATE PROCEDURE EmployeesByExperience()
BEGIN
```

```
   SELECT *
   FROM Employees
   ORDER BY experience DESC;
END //

DELIMITER ;
Call
CALL EmployeesByExperience();
```

---

13. Display Finance Department Employees

Question:

Write a procedure to display Finance department employees.

Answer

```
DELIMITER //

CREATE PROCEDURE FinanceEmployees()
BEGIN
   SELECT *
   FROM Employees
   WHERE department='Finance';
END //

DELIMITER ;
Cal
lCALL FinanceEmployees();
```

---

14. Display HR Employees

Question:

Write a procedure to display all HR employees.

Answer

```
DELIMITER //

CREATE PROCEDURE HREmployees()
BEGIN
   SELECT *
   FROM Employees
   WHERE department='HR';
END //

DELIMITER ;
```

Call

```
CALL HREmployees();
```

---

15. Total Salary Paid

Question:

Write a procedure to calculate the total salary paid to employees.

Answer

```
DELIMITER //

CREATE PROCEDURE TotalSalary()
BEGIN
   SELECT SUM(salary) AS TotalSalary
   FROM Employees;
END //

DELIMITER ;
```

Call

```
CALL TotalSalary();
```

---

16. Employees with Salary Between 50000 and 80000

Question:

Write a procedure to display employees whose salary is between 50000 and 80000.

Answer

```
DELIMITER //

CREATE PROCEDURE SalaryRange()
BEGIN
   SELECT *
   FROM Employees
   WHERE salary BETWEEN 50000 AND 80000;
END //

DELIMITER ;
```

Call

```
CALL SalaryRange();
```

---

17. Employees from Bangalore

Question:

Write a procedure to display employees from Bangalore.

Answer

```
DELIMITER //

CREATE PROCEDURE BangaloreEmployees()
BEGIN
```

```
    SELECT *
    FROM Employees
    WHERE city='Bangalore';
END //

DELIMITER ;
Call
CALL BangaloreEmployees();
```

---

18. Employees Having Salary Above Average

Question:

Write a procedure to display employees whose salary is greater than the average salary.

Answer

```
DELIMITER //

CREATE PROCEDURE AboveAverageSalary()
BEGIN
    SELECT *
    FROM Employees
    WHERE salary > (SELECT AVG(salary) FROM Employees);
END //

DELIMITER ;
Call
CALL AboveAverageSalary();
```

---

19. Department-wise Employee Count

Question:

Write a procedure to display the number of employees in each department.

Answer

```
DELIMITER //

CREATE PROCEDURE DepartmentWiseCount()
BEGIN
   SELECT department,
        COUNT(*) AS TotalEmployees
   FROM Employees
   GROUP BY department;
END //

DELIMITER ;
```

Call

```
CALL DepartmentWiseCount();
```

---

20. Employees with Maximum Experience

Question:

Write a procedure to display the employee(s) having the highest experience.

Answer

```
DELIMITER //

CREATE PROCEDURE HighestExperience()
BEGIN
   SELECT *
   FROM Employees
   WHERE experience = (
      SELECT MAX(experience)
```

```
    FROM Employees
  );
END //

DELIMITER ;
Call
CALL HighestExperience();
```

IN Parameter Procedures

Definition

An IN parameter is used to pass a value into a stored procedure.

The procedure can use the value, but changes made to the parameter inside the procedure are not returned to the caller.

Syntax

```
CREATE PROCEDURE procedure_name(
  IN parameter_name datatype
)
BEGIN
  SQL statements;
END;
```

Simple flow

Value

↓

IN Parameter

↓

Stored Procedure

1. Employee by ID

```
DELIMITER //

CREATE PROCEDURE GetEmployeeById(IN eid INT)
BEGIN
    SELECT * FROM Employees
    WHERE emp_id = eid;
END //

DELIMITER ;
CALL GetEmployeeById(101);
```

---

2. Employees by Department

```
DELIMITER //

CREATE PROCEDURE GetEmployeesByDepartment(IN dept VARCHAR(30))
BEGIN
    SELECT *
    FROM Employees
    WHERE department = dept;
END //

DELIMITER ;
CALL GetEmployeesByDepartment('IT');
```

---

3. Employees by Salary

```
DELIMITER //
```

```
CREATE PROCEDURE GetEmployeesBySalary(IN sal DECIMAL(10,2))
BEGIN
	SELECT *
	FROM Employees
	WHERE salary >= sal;
END //

DELIMITER ;
CALL GetEmployeesBySalary(60000);
```

---

4. Employees by City

```
DELIMITER //

CREATE PROCEDURE GetEmployeesByCity(IN cityName VARCHAR(30))
BEGIN
	SELECT *
	FROM Employees
	WHERE city = cityName;
END //

DELIMITER ;
CALL GetEmployeesByCity('Hyderabad');
```

---

5. Employees by Experience

```
DELIMITER //

CREATE PROCEDURE GetExperiencedEmployees(IN exp INT)
BEGIN
```

```
    SELECT *
    FROM Employees
    WHERE experience >= exp;
END //

DELIMITER ;
CALL GetExperiencedEmployees(5);
```

---

6. Employee by Name

```
DELIMITER //

CREATE PROCEDURE GetEmployeeByName(IN ename VARCHAR(50))
BEGIN
    SELECT *
    FROM Employees
    WHERE emp_name = ename;
END //

DELIMITER ;
CALL GetEmployeeByName('Rahul');
```

---

7. Salary Range

```
DELIMITER //

CREATE PROCEDURE SalaryRange(IN minSal DECIMAL(10,2), IN maxSal DECIMAL(10,2))
BEGIN
    SELECT *
    FROM Employees
```

```
    WHERE salary BETWEEN minSal AND maxSal;
END //

DELIMITER ;
CALL SalaryRange(40000,70000);
```

---

8. Department and Experience

```
DELIMITER //

CREATE PROCEDURE DeptExperience(IN dept VARCHAR(30), IN exp INT)
BEGIN
    SELECT *
    FROM Employees
    WHERE department=dept
    AND experience>=exp;
END //

DELIMITER ;
CALL DeptExperience('IT',5);
```

---

9. City and Salary

```
DELIMITER //

CREATE PROCEDURE CitySalary(IN cityName VARCHAR(30), IN sal DECIMAL(10,2))
BEGIN
    SELECT *
    FROM Employees
    WHERE city=cityName
```

```
        AND salary>=sal;
END //

DELIMITER ;
CALL CitySalary('Hyderabad',60000);
```

---

10. Finance Employees

```
DELIMITER //

CREATE PROCEDURE FinanceEmployees()
BEGIN
        SELECT *
        FROM Employees
        WHERE department='Finance';
END //

DELIMITER ;
CALL FinanceEmployees();
```

OUT Parameter Procedures

Definition

An OUT parameter is used to return a value from the stored procedure to the caller.

The caller provides a variable to receive the result.

```
CREATE PROCEDURE procedure_name(
   OUT parameter_name datatype
)
BEGIN
   SET parameter_name = value;
```

END;

Usually, we call an OUT parameter using a session variable:

```
SET @result = 0;

CALL procedure_name(@result);

SELECT @result;
```

1. Total Employees

```
DELIMITER //

CREATE PROCEDURE TotalEmployees(OUT total INT)
BEGIN
    SELECT COUNT(*) INTO total
    FROM Employees;
END //

DELIMITER ;
CALL TotalEmployees(@t);
SELECT @t;
```

---

2. Maximum Salary

```
DELIMITER //

CREATE PROCEDURE MaximumSalary(OUT maxSal DECIMAL(10,2))
BEGIN
```

```
    SELECT MAX(salary)
    INTO maxSal
    FROM Employees;
END //

DELIMITER ;
CALL MaximumSalary(@m);
SELECT @m;
```

---

3. Minimum Salary

```
DELIMITER //

CREATE PROCEDURE MinimumSalary(OUT minSal DECIMAL(10,2))
BEGIN
    SELECT MIN(salary)
    INTO minSal
    FROM Employees;
END //

DELIMITER ;
```

---

4. Average Salary

```
DELIMITER //

CREATE PROCEDURE AverageSalary(OUT avgSal DECIMAL(10,2))
BEGIN
    SELECT AVG(salary)
    INTO avgSal
```

```
    FROM Employees;
END //

DELIMITER ;
```

---

5. Total Salary

```
DELIMITER //

CREATE PROCEDURE TotalSalary(OUT totalSal DECIMAL(10,2))
BEGIN
    SELECT SUM(salary)
    INTO totalSal
    FROM Employees;
END //

DELIMITER ;
```

---

6. Highest Experience

```
DELIMITER //

CREATE PROCEDURE HighestExperience(OUT exp INT)
BEGIN
    SELECT MAX(experience)
    INTO exp
    FROM Employees;
END //

DELIMITER ;
```

---

7. Lowest Experience

```
DELIMITER //

CREATE PROCEDURE LowestExperience(OUT exp INT)
BEGIN
	SELECT MIN(experience)
	INTO exp
	FROM Employees;
END //

DELIMITER ;
```

---

8. IT Employee Count

```
DELIMITER //

CREATE PROCEDURE ITEmployeeCount(OUT cnt INT)
BEGIN
	SELECT COUNT(*)
	INTO cnt
	FROM Employees
	WHERE department='IT';
END //

DELIMITER ;
```

---

9. Hyderabad Employee Count

```
DELIMITER //
```

```
CREATE PROCEDURE HyderabadCount(OUT cnt INT)
BEGIN
    SELECT COUNT(*)
    INTO cnt
    FROM Employees
    WHERE city='Hyderabad';
END //

DELIMITER ;
```

---

10. Average Experience

```
DELIMITER //

CREATE PROCEDURE AverageExperience(OUT avgExp DECIMAL(5,2))
BEGIN
    SELECT AVG(experience)
    INTO avgExp
    FROM Employees;
END //

DELIMITER ;
```

INOUT Parameter Procedures

INOUT Parameter

Definition

An INOUT parameter is used when we want to send a value into the procedure and receive the modified value back.

In other words:

INOUT = Input + Output

Syntax

```
CREATE PROCEDURE procedure_name(
   INOUT parameter_name datatype
)
BEGIN
   SET parameter_name = new_value;
END;
```

Calling:

```
SET @value = initial_value;

CALL procedure_name(@value);

SELECT @value;
```

Flow

@value

↓

IN

↓

PROCEDURE

↓

modified value

↓

OUT

↓

@value

1. Add Bonus

```
DELIMITER //

CREATE PROCEDURE AddBonus(
        IN empid INT,
        INOUT bonus DECIMAL(10,2)
)
BEGIN
        UPDATE Employees
        SET salary = salary + bonus
        WHERE emp_id = empid;

        SELECT salary
        INTO bonus
        FROM Employees
        WHERE emp_id = empid;
END //

DELIMITER ;
SET @b=5000;
CALL AddBonus(101,@b);
SELECT @b;
```

---

2. Deduct Salary

```
DELIMITER //

CREATE PROCEDURE DeductSalary(
        IN empid INT,
        INOUT amount DECIMAL(10,2)
)
BEGIN
        UPDATE Employees
        SET salary=salary-amount
        WHERE emp_id=empid;

        SELECT salary
        INTO amount
        FROM Employees
        WHERE emp_id=empid;
END //

DELIMITER ;
SET @deduction = 5000;
CALL DeductSalary(101,@deduction);
SELECT @deduction;
```

---

3. Increase Experience

```
DELIMITER //

CREATE PROCEDURE IncreaseExperience(
        IN empid INT,
```

```
        INOUT exp INT
)
BEGIN
        UPDATE Employees
        SET experience=experience+exp
        WHERE emp_id=empid;

        SELECT experience
        INTO exp
        FROM Employees
        WHERE emp_id=empid;
END //

DELIMITER ;
SET @exp = 2;
CALL IncreaseExperience(101,@exp);
SELECT @exp;
```

---

5. Increase Salary by Amount

```
DELIMITER //

CREATE PROCEDURE SalaryIncrement(
        IN empid INT,
        INOUT inc DECIMAL(10,2)
)
BEGIN
```

```
        UPDATE Employees
        SET salary=salary+inc
        WHERE emp_id=empid;

        SELECT salary
        INTO inc
        FROM Employees
        WHERE emp_id=empid;
END //

DELIMITER ;
```

Stored Procedures - Scenario Based Questions (MySQL)

These questions are commonly asked in interviews and practical assessments. Each scenario requires creating a stored procedure.

Scenario 1: Employee Search

Problem

A company wants to search employee details by Employee ID without writing the same query repeatedly.

Task

Create a stored procedure that accepts Employee ID and displays employee details.

Concept: IN Parameter

Scenario 2: Department Employees

Problem

The HR department wants to see all employees working in a specific department.

Task

Create a stored procedure that accepts the department name and displays all employees.

Concept: IN Parameter

Scenario 3: Salary Filter

Problem

The finance team wants to identify employees earning more than a specified salary.

Task

Create a stored procedure that accepts a salary amount and displays all employees earning more than that amount.

Concept: IN Parameter

Scenario 4: City-wise Employees

Problem

The company has offices in multiple cities. Management wants to view employees from a particular city.

Task

Create a stored procedure that accepts a city name and displays employees from that city.

Concept: IN Parameter

Scenario 5: Experience-Based Employees

Problem

The recruitment team wants employees who have at least a certain number of years of experience.

Task

Create a stored procedure that accepts years of experience and displays matching employees.

Concept: IN Parameter

Scenario 6: Total Employees

Problem

The CEO wants to know the total number of employees in the company.

Task

Create a stored procedure that returns the total employee count.

Concept: OUT Parameter

Scenario 7: Highest Salary

Problem

The finance department wants to know the highest salary paid in the company.

Task

Create a stored procedure that returns the highest salary.

Concept: OUT Parameter

Scenario 8: Average Salary

Problem

The accounts department wants to calculate the average salary of all employees.

Task

Create a stored procedure that returns the average salary.

Concept: OUT Parameter

Scenario 9: Total Salary Expenditure

Problem

Management wants to know the total monthly salary expenditure.

Task

Create a stored procedure that returns the sum of all employee salaries.

Concept: OUT Parameter

Scenario 10: IT Employee Count

Problem

The IT manager wants to know how many employees are working in the IT department.

Task

Create a stored procedure that returns the number of IT employees.

Concept: OUT Parameter

Scenario 11: Bonus Distribution

Problem

The company announces a ₹5,000 bonus for a specific employee.

Task

Create a stored procedure that accepts Employee ID and Bonus Amount, updates the salary, and returns the updated salary.

Concept: INOUT Parameter

Scenario 12: Salary Increment

Problem

Every year, employees receive an increment.

Task

Create a stored procedure that accepts Employee ID and Increment Amount, updates the salary, and returns the new salary.

Concept: INOUT Parameter

Scenario 13: Tax Deduction

Problem

The payroll department deducts tax from employees' salaries.

Task

Create a stored procedure that accepts Employee ID and Tax Amount, deducts the tax, and returns the updated salary.

Concept: INOUT Parameter

Scenario 14: Promotion

Problem

An employee gets promoted, resulting in both a salary increase and an increase in experience.

Task

Create a stored procedure that updates the employee's salary and experience and returns the updated salary.

Concept: INOUT Parameter

Scenario 15: Festival Bonus

Problem

During a festival, employees receive an additional bonus.

Task

Create a stored procedure that adds the bonus to the salary and returns the revised salary.

Concept: INOUT Parameter

Scenario 16: Student Results

Problem

A college wants to display all students who scored above a specified mark.

Task

Create a stored procedure that accepts the minimum marks and displays qualifying students.

Concept: IN Parameter

Scenario 17: Product Search

Problem

An e-commerce website allows users to search products by category.

Task

Create a stored procedure that accepts the category name and displays matching products.

Concept: IN Parameter

Scenario 18: Order Summary

Problem

An online shopping system wants to know the total number of orders placed.

Task

Create a stored procedure that returns the total number of orders.

Concept: OUT Parameter

Scenario 19: Bank Account Deposit

Problem

A customer deposits money into a bank account.

Task

Create a stored procedure that accepts Account Number and Deposit Amount, updates the balance, and returns the updated balance.

Concept: INOUT Parameter

Scenario 20: Hospital Patient Search

Problem

A hospital receptionist wants to retrieve patient details using the Patient ID.

Task

Create a stored procedure that accepts Patient ID and displays the patient's information.

Concept: IN Parameter

Scenario 21: Library Management

Problem

A librarian wants to know how many books are currently available.

Task

Create a stored procedure that returns the total number of available books.

Concept: OUT Parameter

Scenario 22: Hotel Room Booking

Problem

A hotel receptionist books a room for a customer.

Task

Create a stored procedure that updates the room status from Available to Booked and returns the updated status.

Concept: INOUT Parameter

Scenario 23: Inventory Management

Problem

A store sells a product, reducing the available stock.

Task

Create a stored procedure that accepts Product ID and Quantity Sold, updates the stock, and returns the remaining stock.

Concept: INOUT Parameter

Scenario 24: Employee Transfer

Problem

An employee is transferred to another department.

Task

Create a stored procedure that accepts Employee ID and New Department, updates the department, and displays the updated employee details.

Concept: IN Parameter

Scenario 25: Payroll System

Problem

The payroll department wants to identify employees whose salary falls within a specified range.

Task

Create a stored procedure that accepts a minimum salary and maximum salary and displays all matching employees.

Concept: IN Parameter

Summary

Scenario	Concept
Employee Search	IN
Department Employees	IN
Salary Filter	IN
City-wise Employees	IN
Experience Search	IN
Total Employees	OUT
Highest Salary	OUT
Average Salary	OUT
Total Salary	OUT
IT Employee Count	OUT

Bonus Distribution	**INOUT**
Salary Increment	**INOUT**
Tax Deduction	**INOUT**
Promotion	**INOUT**
Festival Bonus	**INOUT**
Student Results	**IN**
Product Search	**IN**
Order Summary	**OUT**
Bank Deposit	**INOUT**
Hospital Search	**IN**
Library Books Count	**OUT**
Hotel Booking	**INOUT**
Inventory Update	**INOUT**
Employee Transfer	**IN**
Payroll Salary Range	**IN**